Assignment 1

The assignment consists of two parts: a mandatory project that evolves over the entire course, and a set of choose-yourself exercises. You need **100+ points** to pass an assignment and can get up to **50** points for your mandatory project solution - the remaining points must come from the additional exercises you choose to solve. Annotated points correspond to exercises solved with nothing left for your TA to wish for, so it may be a good idea to aim for more than a 100 points total.

We recommend reading the assignment through and decide which exercises you find most interesting before you start.

How to hand in

For each assignment, students are expected to `.zip` and hand in two folders: one for the mandatory project and one for the exercises you chose to solve. Include all necessary files so your code immediately runs on your TAs computer as well. We recommend making a `DatF/` folder at course start with subfolders `MandatoryProject/`, `Exercises1/`, `Exercises2/`, and so forth. Every project and exercises folder should also contain a `.git` history as explained at the lectures. Make sure to commit changes often and explain what you did. **Happy coding!**

Minimum requirements for getting points

- Manage your `datf` environment using `conda` (e.g. for installing new packages you may need)
- Initialize your project and exercises folders as Git repositories and commit changes *frequently*. You can only recover and document your work as detailed as you commit to the Git history.
- Use *relative paths* when you read or write files. Your code won't run on your TAs laptop if it references *your* file system.

Contents

1 Mandatory (50 points) **3**
1.1 Building your programming setup 3
1.2 DatF Project - Part 1 3

2 Exercises **4**
2.1 Loops & Conditionals 4
2.2 Input / Output 5
2.3 Fitting 5
2.4 Simulation 6

1 Mandatory (50 points)

1.1 Building your programming setup

Complete the setup of course tools as explained in the lectures:

- **Visual Studio Code** (Editor)
- **Terminal usage** (Run code and use all the tools in one place)
- **Anaconda** (Managing python environments)
- **Git** (Document, collaborate, version control, and share your code)

See the **Setup Guide** for a step-by-step walk through. Hand in a screenshot showing that you can run '`conda --version`' and '`git --version`' in the VS Code terminal.

1.2 DatF Project - Part 1

Download the `MandatoryProjectPart1/` folder from the **Course Page** containing the Jupyter notebook `error-propagation.ipynb` and a `media/` folder. Put the contents in your `MandatoryProject/` folder and initialize it as a Git repository. Go through the notebook step-by-step and solve the Tasks when prompted. Commit to the Git history every time you add new contents. Every Task must be solved in order to pass Part 1. Hand in your `MandatoryProject/` folder with your additions to the notebook.

OBS: the `media/` folder must stay next to the notebook for illustrations to render.

2 Exercises

Data files referenced in some of the exercises are available for download at the **Course Page**. Answers for any exercise must adhere to the Minimum Requirements which may evolve throughout the course.

2.1 Loops & Conditionals

Question 1 — 10 points

Finding Your Prime

You can earn upwards of $150,000 for discovering a new prime number with over 10^8 digits. Let's start somewhere smaller:

Your task is to write a small program that finds all the prime numbers below 100 and saves it to a `list`. Using `for / while`, and `if`, go through each number, check whether it has any factors — if not, collect it as a prime **(8 points)**. Finally, print the list and verify that the numbers are indeed all the primes below 100 **(2 points)**.

Tip: Check out the modulo operator (`%`)

Concepts of Programming: Flow Control

Question 2 — 10 points

The Collatz Conjecture

The Collatz conjecture states that any positive integer will eventually reach 1 if we repeatedly apply the following rules:

- If the number is **even**, divide it by 2.
- If the number is **odd**, multiply it by 3 and add 1.

For example, starting from 6, we obtain:

$$6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$$

Despite its simplicity, the conjecture remains mathematically unproven!

Your Task:

Write a Python program that tests the conjecture for every integer from 1 to 100. For each starting number, count how many steps it takes to reach 1.

Questions:

- Which starting number between 1-100 requires the most steps and how many? **(5 points)**
- Across all starting numbers, what is the highest integer we ever encounter? (I.e. starting at 6 we reached a maximum of 16) **(5 points)**

Concepts of Programming: Flow Control

2.2 Input / Output

Question 3 — 10 points

Particle Detector Event Selection
At a particle collider, physicists study collisions between high-energy particles. Each collision may produce several particles, which are recorded by detectors. Since millions of collisions can occur every second, physicists rely on computer programs to identify events of interest. Imagine that you are working as a physicist at CERN, developing a simple program to filter collision events based on the particles detected. For simplicity, we represent each detected particle by a single letter:

- e — Electron
- m — Muon
- p — Proton
- g — Photon (gamma ray)

The file `detector_events.txt` contains 100 simulated collision events, each with a selection criterion specifying how many particles of a particular type must be detected for the event to be accepted. Each line contains a selection criterion, a colon (:), and a sequence of detected particles:

```
1-3 g:  geppg
2-3 e:  gpep
```

The criterion `1-3 g:` means that the event must contain at least 1 and at most 3 photons (g) to be accepted. In the example above:

- The first event is accepted, since it contains 2 photons.
- The second event is rejected, since it contains only 1 electron, but requires at least 2.

Your Task: Write a Python program that reads `detector_events.txt` **(4 points)**, examines each collision event, and determines how many events satisfy their respective selection criteria. Print the total number of accepted events **(6 points)**.

OBS: Your program should work regardless of the number of particles recorded in each event.

Concepts of Programming: Data Structures, Flow Control

2.3 Fitting

Question 4 — 20 points

Streglisten
The four .csv files `stregliste1-4` contain the actual(!!), uncleaned (but anonymized) data on the drinking habits during RUSTUR for the last four years.

Plot a histogram of "number of beers per person" - regardless of brand **(8 points)**. Fit number of beers per person with a combined exponential decay and log-normal distribution. Make a figure showcasing your fit on top of the histogram **(8 points)**. How would you interpret the data and fitted distribution? **(4 points)**

Bonus: Can you find something else interesting in the dataset?

TIP: check out `pandas` for handling .csv files and `scipy.optimize.curve_fit` for fitting.

Concepts of Programming: Data Structures

2.4 Simulation

Question 5 — 50 points

Emergent Dynamics: Conway's Game of Life

In physics, complex collective phenomena can sometimes emerge from surprisingly simple local interactions. Examples include magnetic ordering, crystal growth, and the propagation of waves. To explore this idea, we will investigate a simple **cellular automaton**: Conway's Game of Life. Imagine a two-dimensional grid where each site can be in one of two states:

'#' — Active (occupied)

'.' — Inactive (empty)

Each site interacts with its **8 neighboring sites**. At every time step, all sites are updated *simultaneously*, according to three rules:

- **Survival:** An active site remains active if it has exactly 2 or 3 active neighbors. Otherwise, it becomes inactive.
- **Activation:** An inactive site becomes active if it has exactly 3 active neighbors.
- **Otherwise:** An inactive site remains inactive.

The grid has fixed boundaries: positions outside the grid are always considered inactive.

Your Task:

Write a Python program that simulates the evolution of this system.

1. Read an initial grid configuration from a `.txt` file, using the #/. notation above.
2. Implement the update rules so that all sites evolve simultaneously. **(30 points)**
3. Print the initial configuration and its evolution at selected time steps. **(8 points)**
4. Record the total number of active sites, $N(t)$, at every time step and plot it. **(8 points)**

You will be given three initial configurations, named `blinker.txt`, `glider.txt`, and `pentomino.txt`. Run each configuration for at least 16 time steps.

Questions: (4 points)

- Does every configuration eventually become stationary?
- Can a localized structure move through the grid even though individual sites never move?
- Is the total number of active sites conserved by the update rules?

Fun fact: Despite its simple rules, Conway's Game of Life can generate remarkably complex dynamics. This makes it a classic example of *emergence*: collective behavior that is not immediately obvious from the rules governing individual components. In fact, it turns out that Game of Life is rich enough to even simulate Game of Life *inside itself!* (**video**)

Concepts of Programming: Data Structures, Flow Control

Question 6 — 10 points

Game of Life continued - solving any of these bonus tasks gives additional 10 points each

- Collect the state of the grid at every time point and turn it into an animation (.gif). Checkout matplotlib's `FuncAnimation`.
- Make your own initial condition `.txt` file and describe its evolution in time. Can you get something to self-replicate?
- Implement 2 different 'active species' and a new rule for how they interact. Can you get interesting competition dynamics?

End of the assignment.